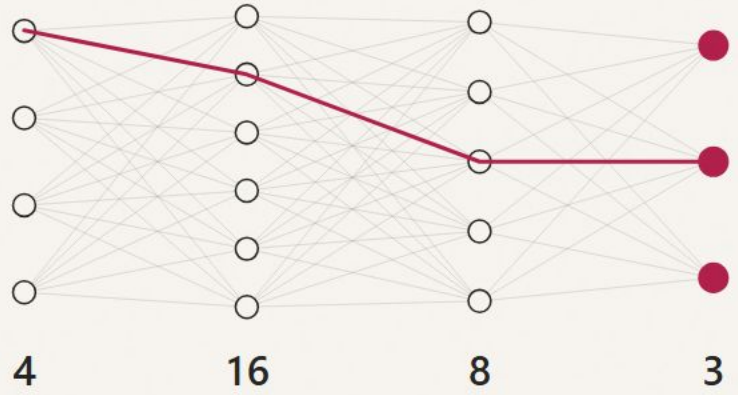
Make learning work.

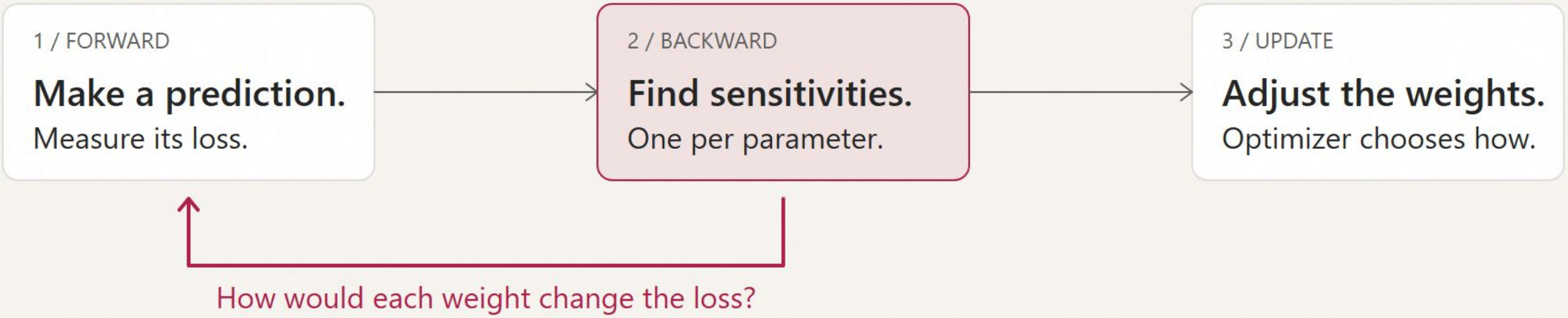
Anush Sankaran
Microsoft Research Scientist
AI Fluency Labs

Recall backprop. Choose the update.
Keep the signal alive. Make training work.



90 MINUTES / ONE RUNNING EXAMPLE

Backprop finds sensitivities, not new weights.



Forward: values. Backward: gradients. Update: new parameters.

A gradient measures the effect of a small nudge.

ONE WEIGHT: THE LOCAL SLOPE

$$g = \frac{\partial L}{\partial w}$$

$$\Delta L \approx g \cdot \Delta w$$

Positive slope? A small decrease in w lowers L.

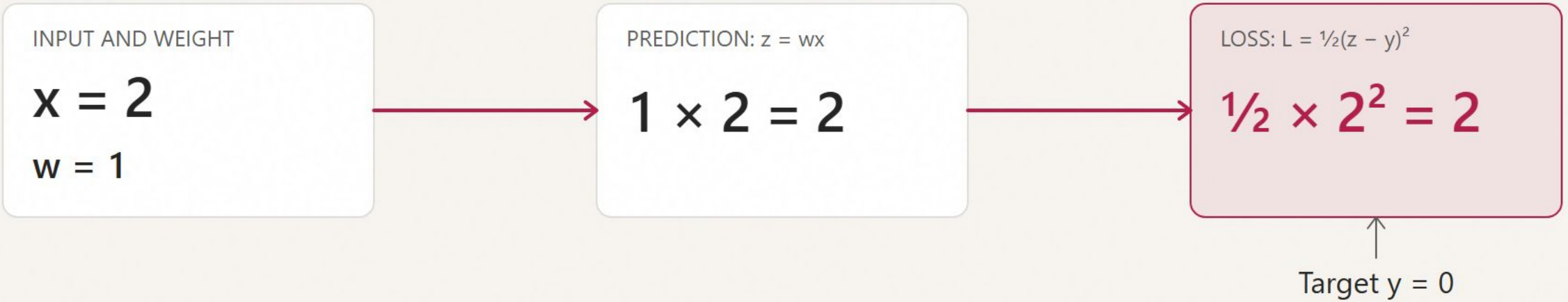
$$\nabla L = (\partial L/\partial w_1, \partial L/\partial w_2, \ldots)$$

Many weights: collect all their slopes.



A gradient is local sensitivity—not the error, a probability, or a step size.

First, run the numbers forward.



Illustrative scalar exercise. Keep the forward values for the backward pass.

Start backward at the loss.

$$L = \tfrac{1}{2}(z - y)^2$$

$$\frac{\partial L}{\partial z} = z - y = 2$$

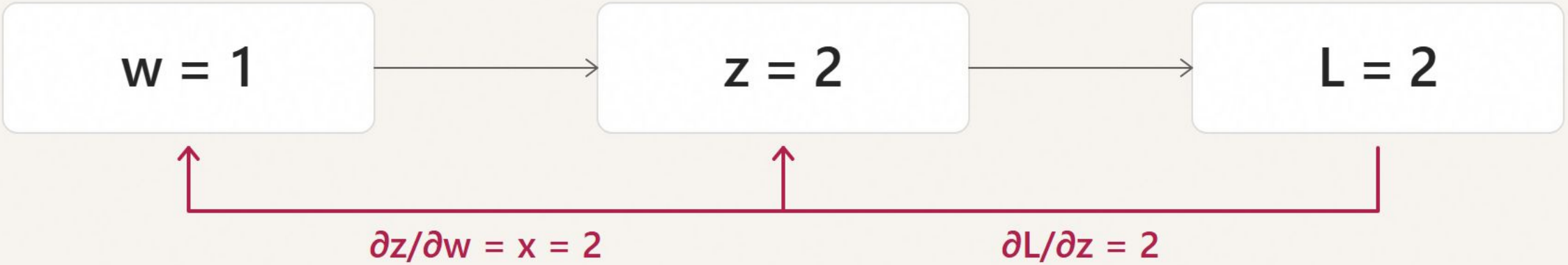
$$\Delta z = +0.01$$

$$\Delta L \approx 2 \times 0.01 = +0.02$$



"If the prediction increases a little, how much does the loss increase?"

The chain rule multiplies local sensitivities.



$$\partial L/\partial w = (\partial L/\partial z) \times (\partial z/\partial w) = 2 \times 2 = 4$$

Incoming sensitivity × local derivative = sensitivity to the earlier value.

Only the optimizer changes the weight.

BACKPROP HAS FINISHED: G = 4

$$w' = w - \eta g$$
$$= 1 - 0.1 \times 4 = 0.6$$

$$z' = 0.6 \times 2 = 1.2$$
$$L' = \tfrac{1}{2} \times 1.2^2 = 0.72$$

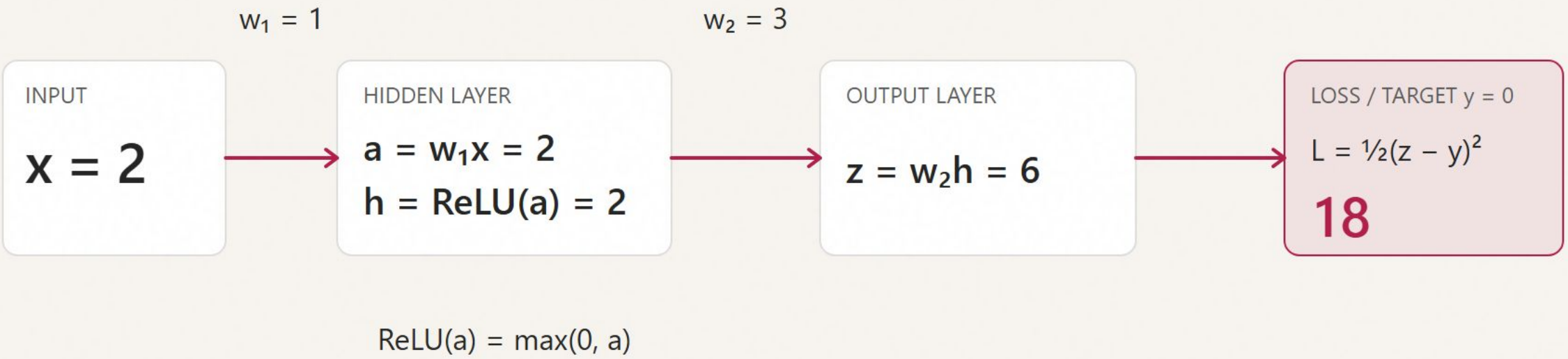
BEFORE

w = 1

Loss = 2

Here η = 0.1 is a chosen learning rate—not a result of backpropagation.

More layers simply add more links.



$\text{ReLU}(a) = \max(0, a)$

Reset: $w_1 = 1$, $w_2 = 3$; no biases. Save $a = 2$ and $h = 2$. Do not update yet.

Each layer computes and passes a gradient.

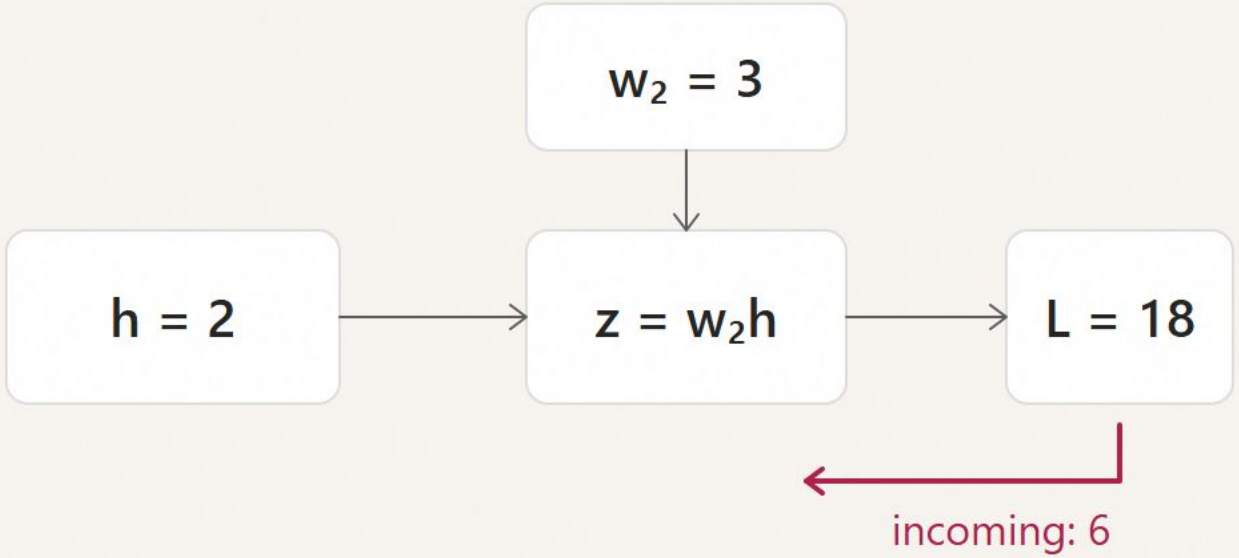
$$\frac{\partial L}{\partial z} = z - y = 6$$

FOR ITS OWN WEIGHT

$$\frac{\partial L}{\partial w_2} = 6 \times h = 6 \times 2 = 12$$

FOR THE EARLIER LAYER

$$\frac{\partial L}{\partial h} = 6 \times w_2 = 6 \times 3 = 18$$



The same incoming sensitivity answers two different questions.

The earlier layer repeats the same rule.

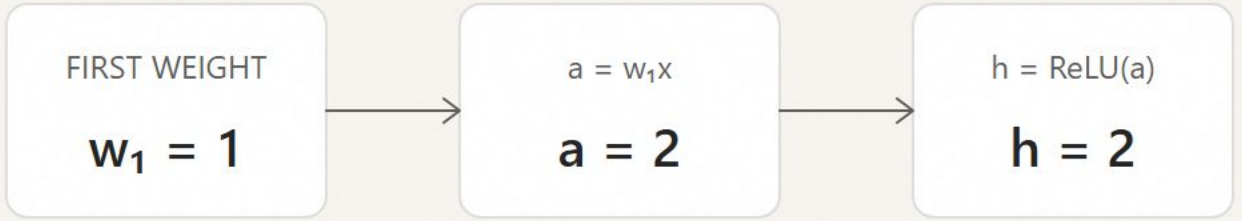
INCOMING SENSITIVITY: ∂L/∂H = 18

$$\frac{\partial L}{\partial a} = 18 \times \mathrm{ReLU}'(2) = 18$$

$$\frac{\partial L}{\partial w_1} = 18 \times x = 36$$

$\nabla L = (36, 12)$

Order: (w_1, w_2). Both weights are still unchanged.



Activation derivatives are part of the chain—not a separate learning algorithm.

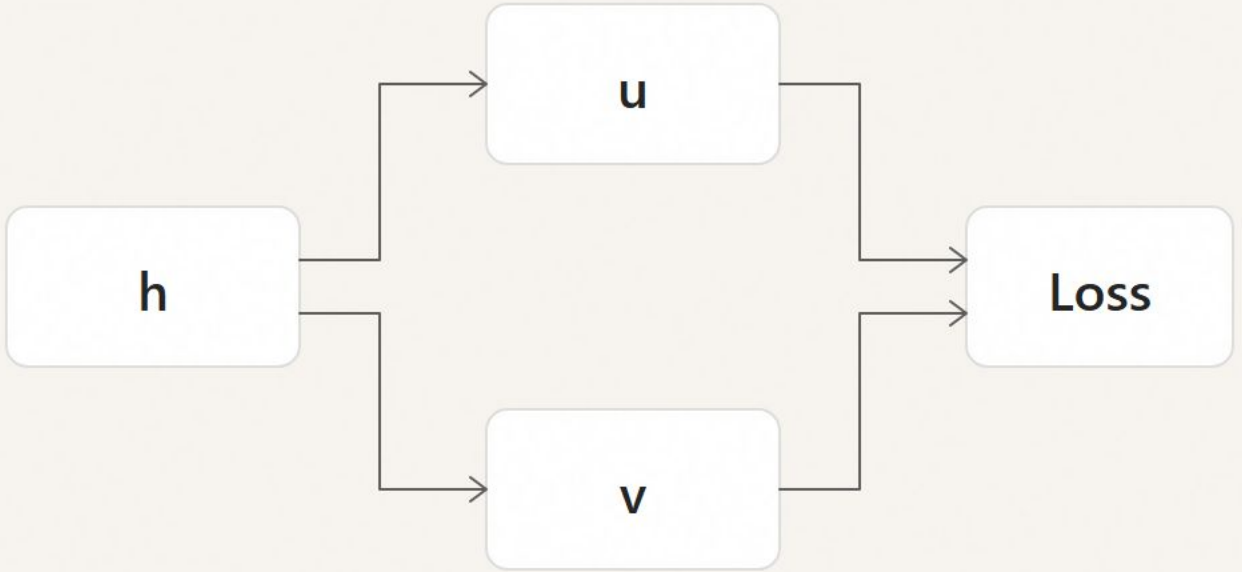
Multiply along a path. Add across paths.

H INFLUENCES THE LOSS THROUGH U AND V

$$g_u = (\partial L/\partial u)(\partial u/\partial h)$$

$$g_v = (\partial L/\partial v)(\partial v/\partial h)$$

$$\frac{\partial L}{\partial h} = g_u + g_v$$



Dense connections, skip connections, and shared weights all need gradient accumulation.

A whole layer follows the same recipe.

$$a = Wx + b, \quad h = \varphi(a)$$

1 / CROSS THE ACTIVATION

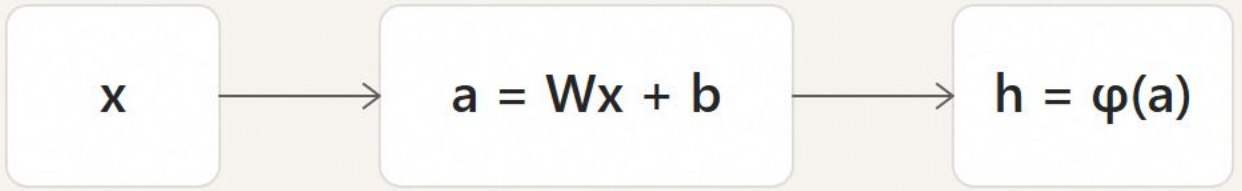
$$\delta = (\partial L/\partial h) \odot \varphi'(a)$$

2 / SAVE PARAMETER GRADIENTS

$$\partial L/\partial W = \delta x^T, \quad \partial L/\partial b = \delta$$

3 / PASS SENSITIVITY BACKWARD

$$\partial L/\partial x = W^T \delta$$



One example; column vectors. ⊙ = elementwise multiply; T = transpose. Repeat for every layer.

Each layer builds a new feature vector.

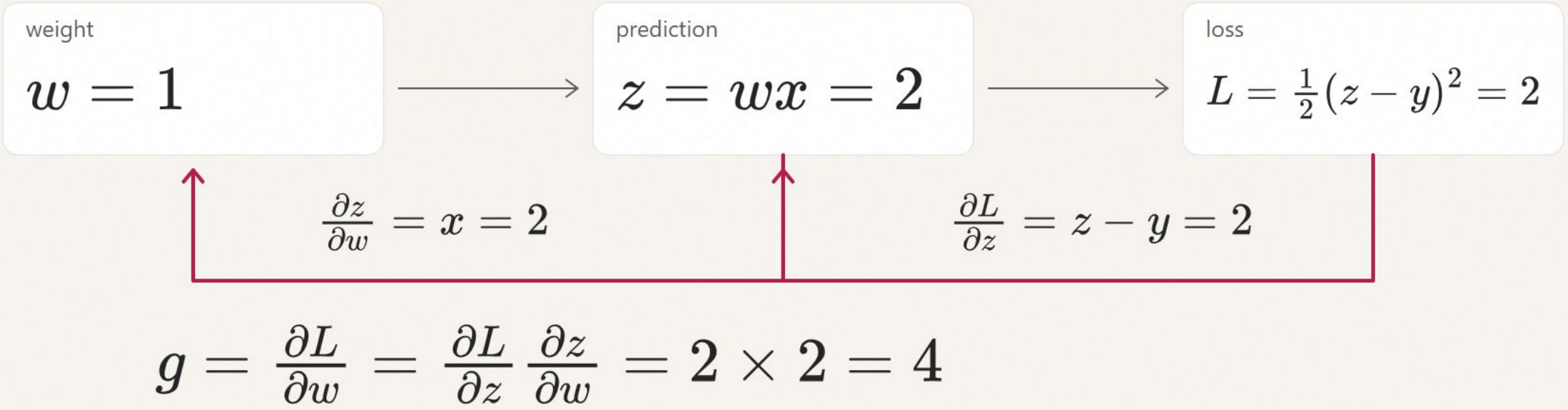
SUPPLIED FEATURES



$$x \in \mathbb{R}^4 \qquad h_1 = \mathrm{ReLU}(W_1 x + b_1) \qquad h_2 = \mathrm{ReLU}(W_2 h_1 + b_2) \qquad z = W_3 h_2 + b_3$$

Each cell is a coordinate—not its numerical value.

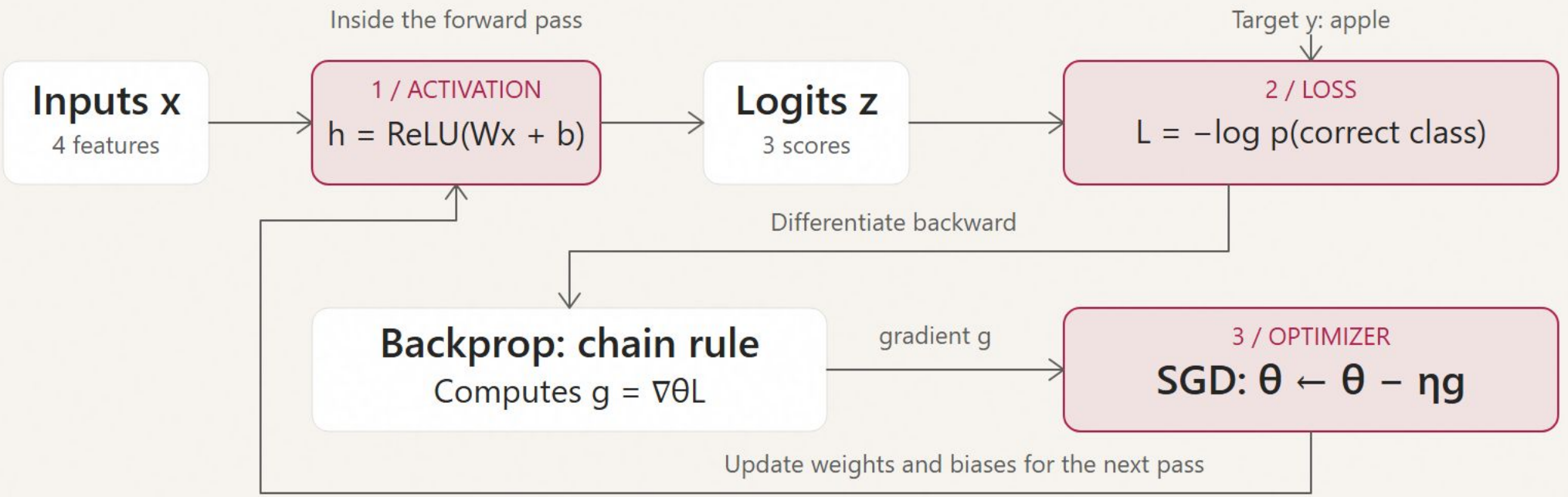
Backprop carries sensitivity backward.



$$g = \frac{\partial L}{\partial w} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial w} = 2 \times 2 = 4$$

Scalar exercise: x = 2, y = 0; squared error, not fruit classification.

Three functions. Three different jobs.



Give every feature a sensible ruler.

$$\widetilde{x}_j = \frac{x_j - \mu_{\text{train},j}}{\sigma_{\text{train},j} + \epsilon}$$

RAW MEASUREMENTS

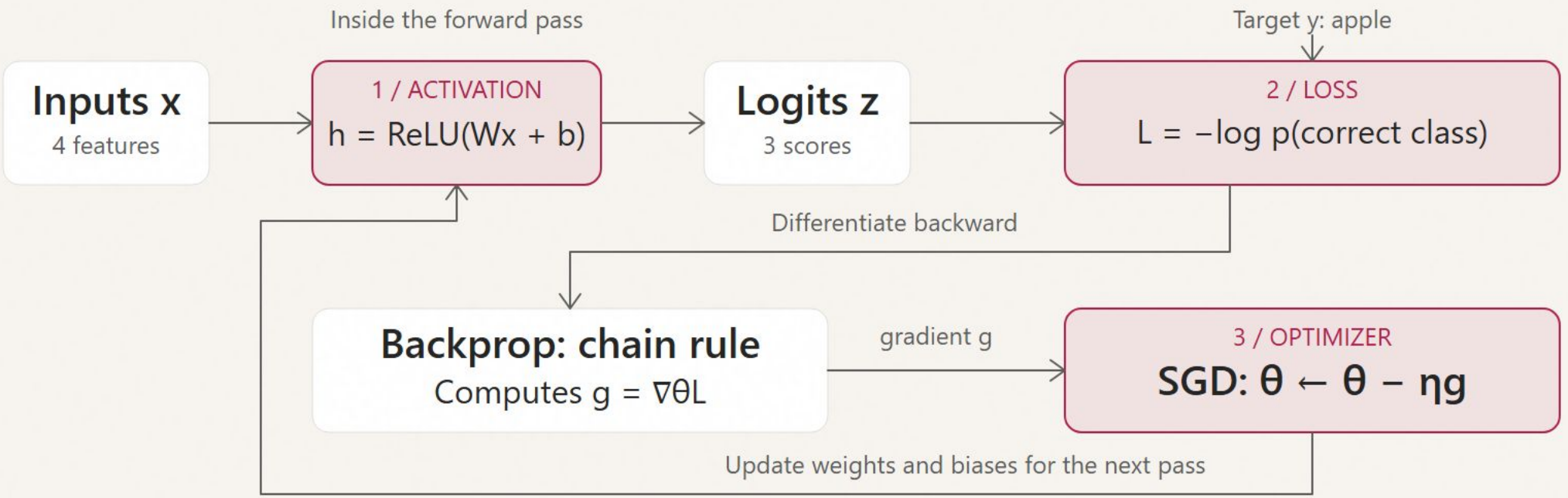
$$[150,\ 7,\ 7,\ 6]$$

STANDARDIZED

$$[1,\ -1,\ 1,\ 0.5]$$

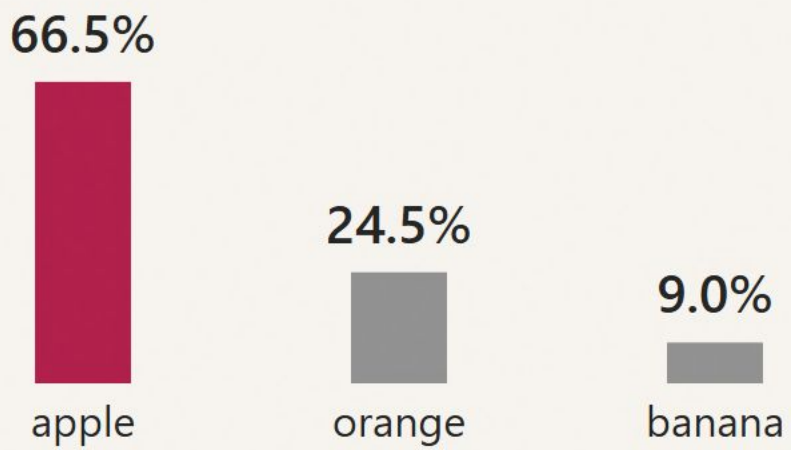
Illustrative μ = [120, 10, 5, 5]; σ = [30, 3, 2, 2]. Reuse training statistics.

Three functions. Three different jobs.



Reward probability on the correct class.

LOGITS: [2, 1, 0]

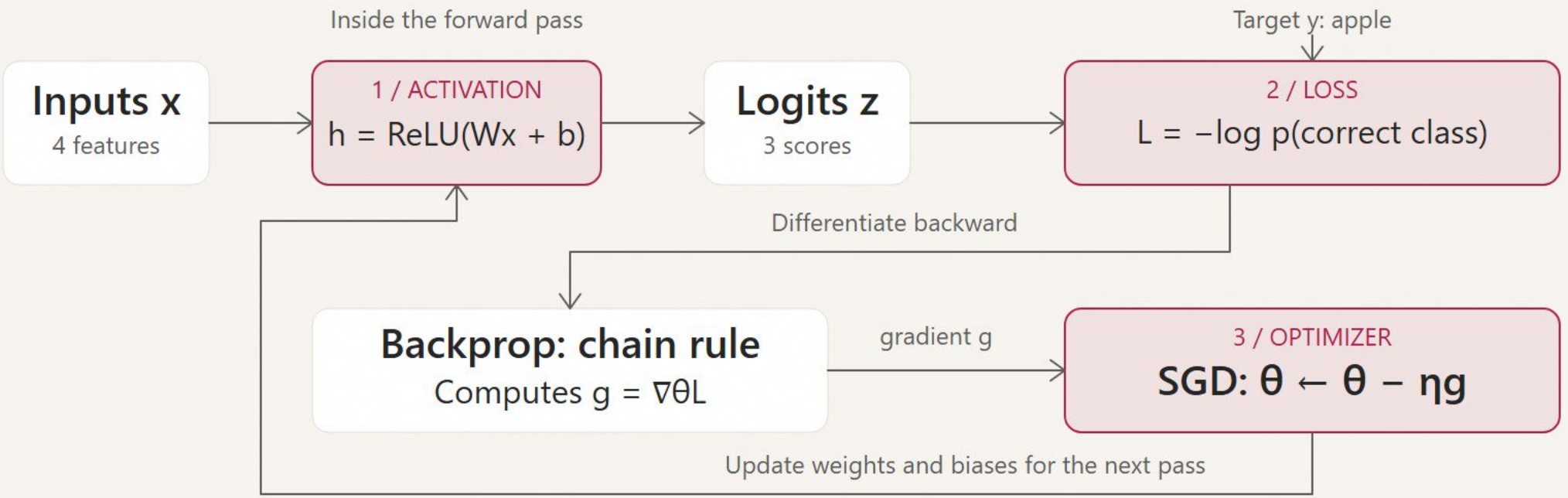


$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

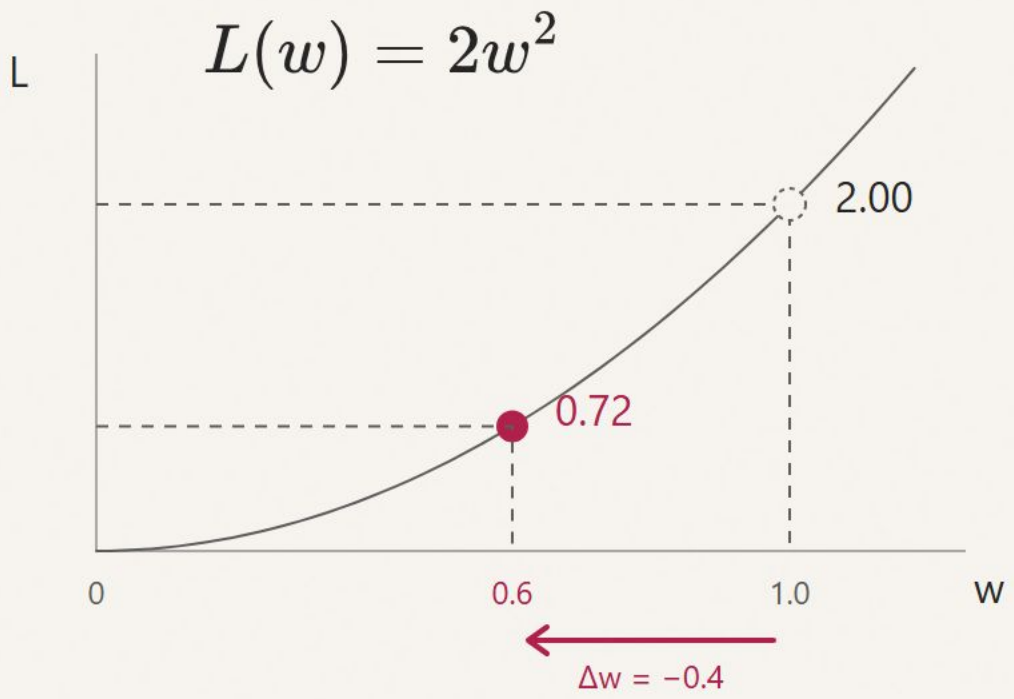
$$L = -\log p_y \approx 0.408$$

> CrossEntropyLoss takes **logits**.

Three functions. Three different jobs.



The optimizer turns sensitivity into a move.



CHOOSE THE STEP SIZE

$$g = 4, \quad \eta = 0.1$$

$$w_{\text{new}} = w - \eta g$$

$$1 - 0.1 \times 4 = 0.6$$

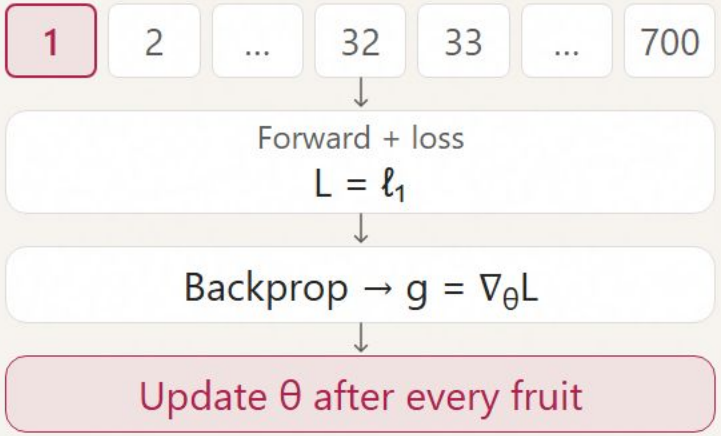
Recompute: loss **2.00** → **0.72**

One illustrative SGD step—not a guarantee.

Same backprop. Different update rhythm.

One sample

B = 1 · stochastic gradient descent

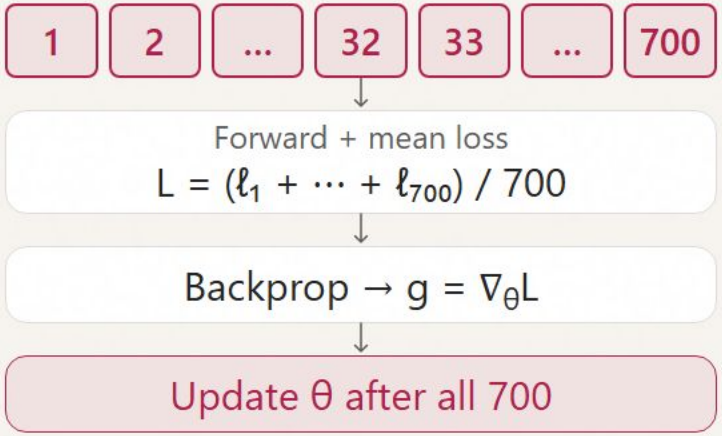


700 updates / epoch

700 groups × 1 fruit

Entire batch

B = 700 · full-batch gradient descent

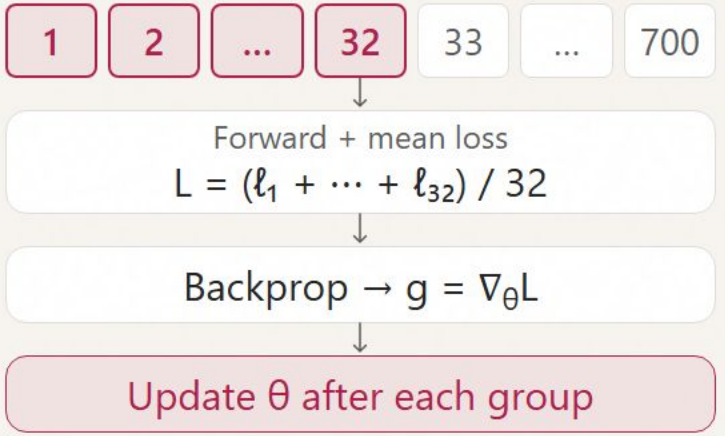


1 update / epoch

1 group × 700 fruits

Mini-batch

B = 32 · mini-batch gradient descent



22 updates / epoch

21 groups × 32 + final 28 fruits

First update highlighted · Same 700 fruits · Backprop computes gradients; the optimizer changes weights.

The whole loop fits on one screen.

```
model.train()
for x, y in train_loader:
    optimizer.zero_grad(set_to_none=True)
    logits = model(x)
    loss = criterion(logits, y)
    loss.backward()
    optimizer.step()
```

Compute.

Differentiate.

Update.

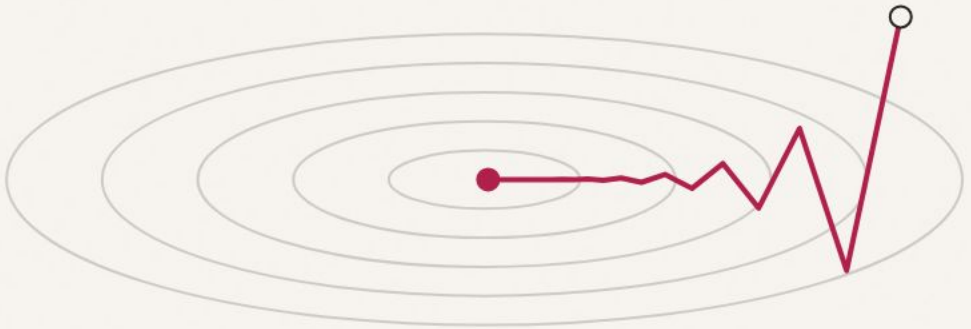
SGD follows the current slope.

Baseline: use the current batch gradient, with one global learning rate.

$$g_t = \nabla_\theta L_{B_t}(\theta_{t-1})$$

$$\theta_t = \theta_{t-1} - \eta g_t$$

No history. No per-coordinate rescaling.



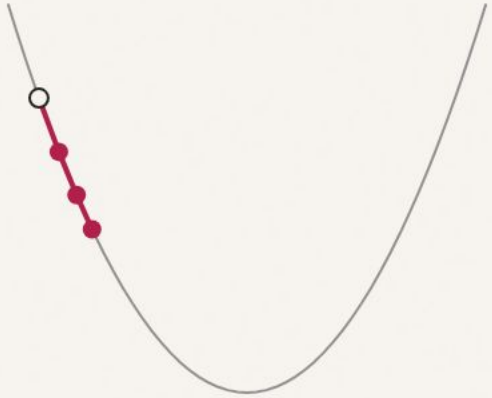
$\eta = 0.13$

$\theta_1 \rightarrow$

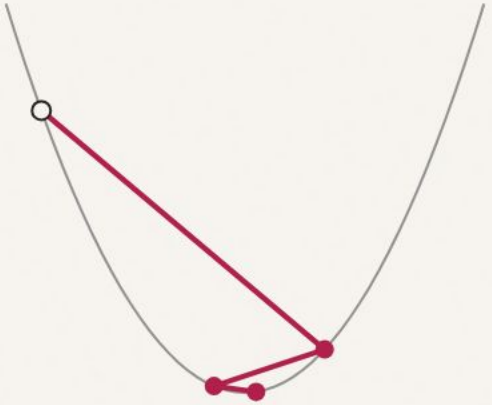
Challenge: narrow valleys cause zigzags; mini-batches also add noise.

The learning rate sets your stride.

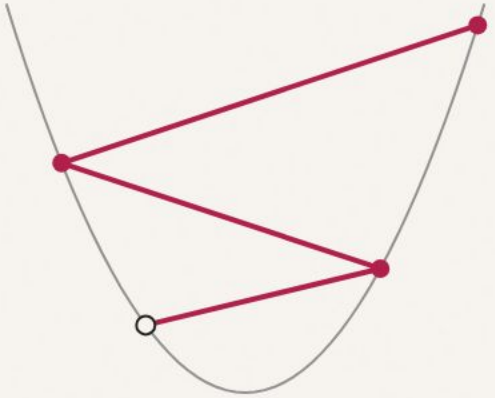
TOO SMALL



USEFUL



TOO LARGE



Illustrative quadratic. Direction does not tell you a safe distance.

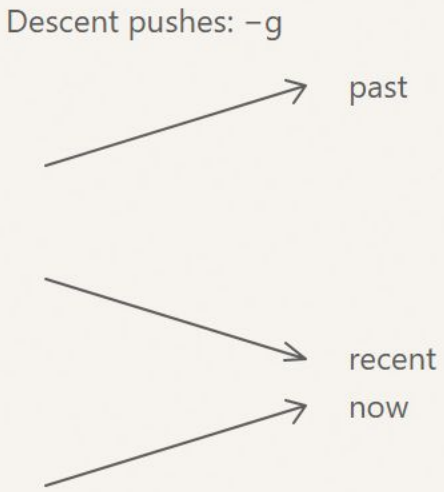
Momentum remembers direction.

SGD can waste updates reversing across a narrow valley.

$$v_t = \beta v_{t-1} + g_t$$

$$\theta_t = \theta_{t-1} - \eta v_t$$

$v_0 = 0 \cdot \beta = 0$ recovers vanilla SGD.



Still limited: momentum can overshoot; one η does not adapt each coordinate.

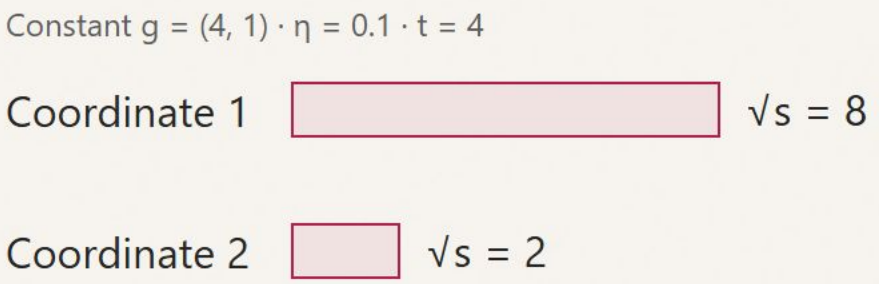
AdaGrad adapts each coordinate.

Momentum adds memory, but not a separate scale for each parameter.

$$s_t = s_{t-1} + g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{s_t} + \varepsilon}$$

$s_0 = 0$ · element-wise squares and division.



Useful for sparse gradients; the lifetime tally can make later steps too small.

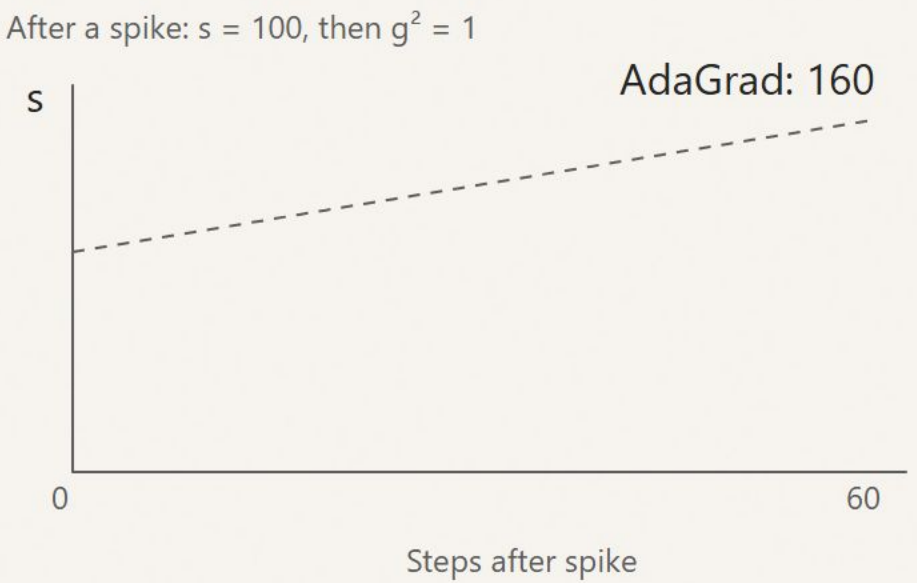
RMSProp forgets stale scale.

AdaGrad never forgets: its accumulated denominator cannot decrease.

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2$$

$$\theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{s_t} + \varepsilon}$$

Exponential average · usual initialization $s_0 = 0$.



Recent scale adapts; the numerator still uses only the current gradient.

Adam combines direction and scale.

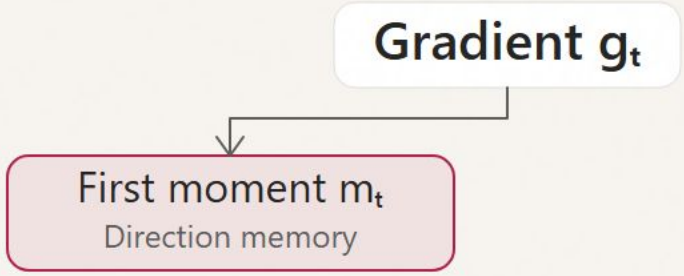
Momentum remembers direction; RMSProp adapts scale. Use both ideas.

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

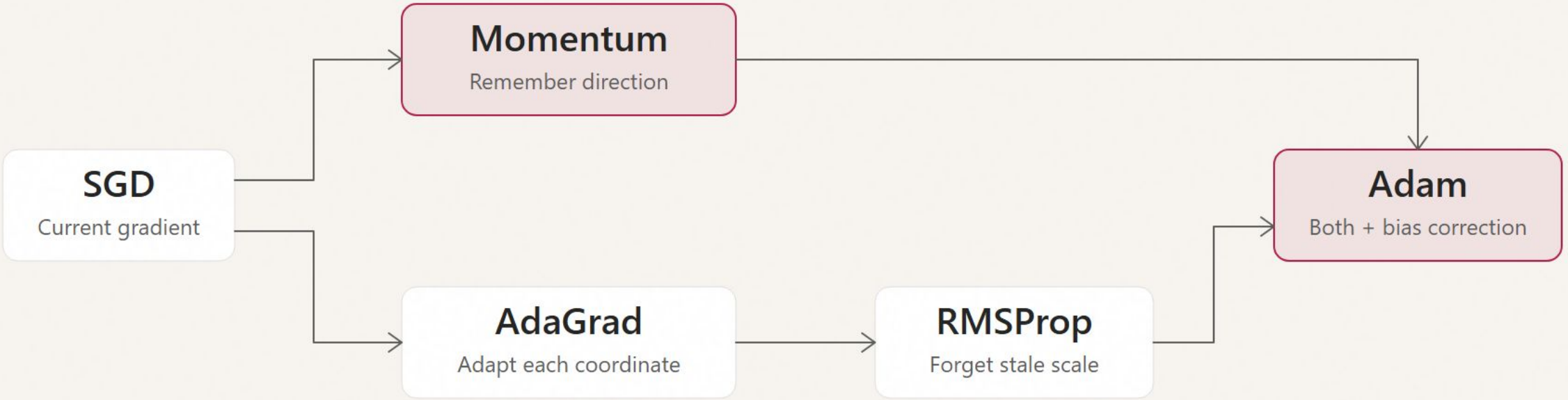
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$



$m_0 = v_0 = 0 \cdot t$ starts at 1 · still tune η and monitor validation.

Two ideas meet in Adam.



Conceptual relationships—not a historical replacement ladder or a winner ranking.

Same landscape. Different walking styles.

SGD



$\eta = 0.13$ $\theta_1 \rightarrow$

MOMENTUM



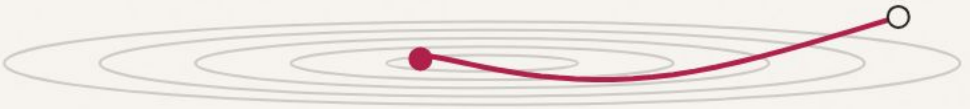
$\eta = 0.055$ $\theta_1 \rightarrow$

RMSPROP



$\eta = 0.17$ $\theta_1 \rightarrow$

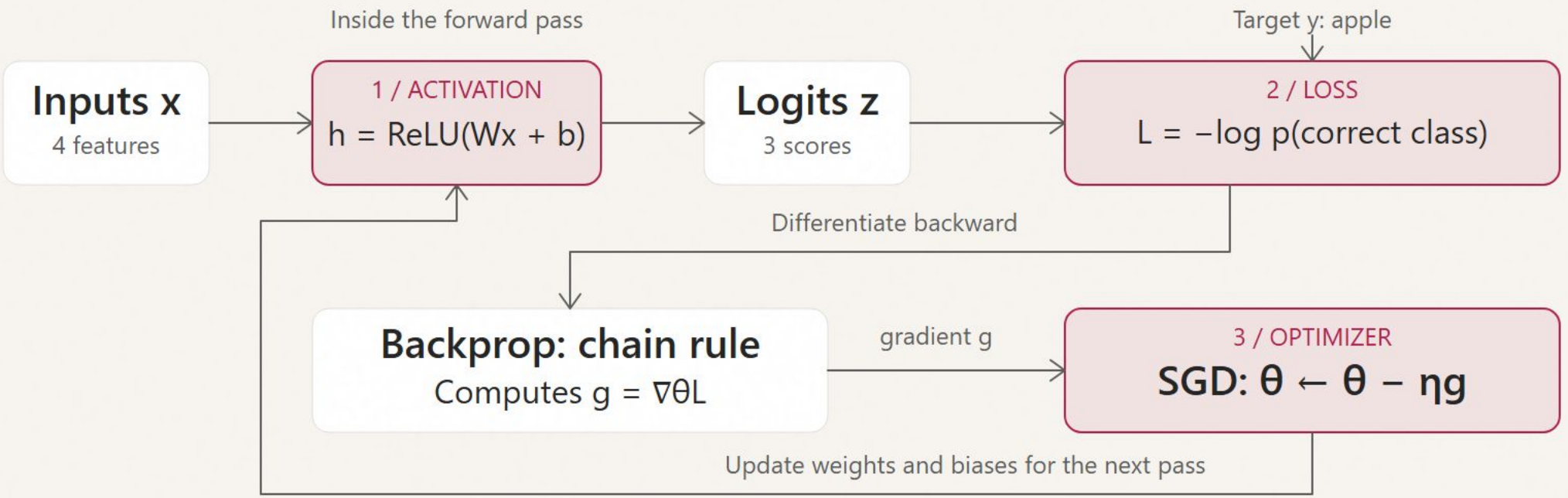
ADAM



$\eta = 0.15$ $\theta_1 \rightarrow$

Computed quadratic · 34 steps · distinct learning rates · not a ranking

Three functions. Three different jobs.



Nonlinearity makes depth useful.

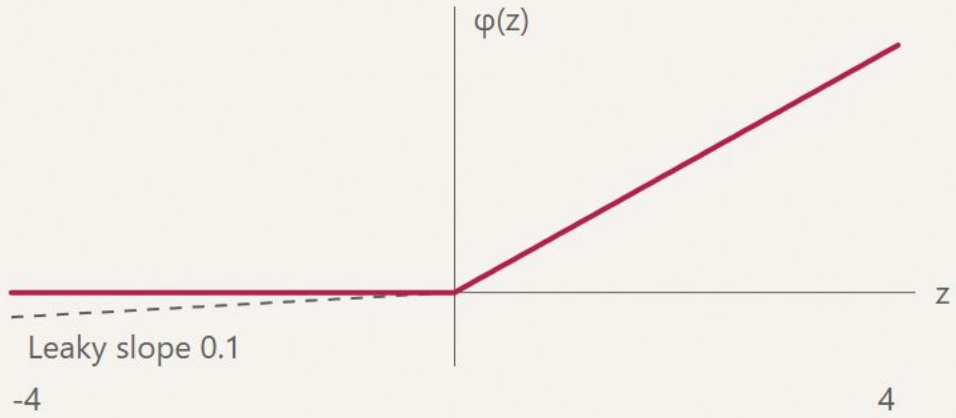
An optimizer cannot replace the activation inside the forward pass.

$$h = \varphi(Wx + b)$$

Without φ: stacked affine layers collapse to one.

$$\mathrm{ReLU}(z) = \max(0, z)$$

$\mathrm{ReLU}(-2) = 0$ · $\mathrm{ReLU}(2) = 2$

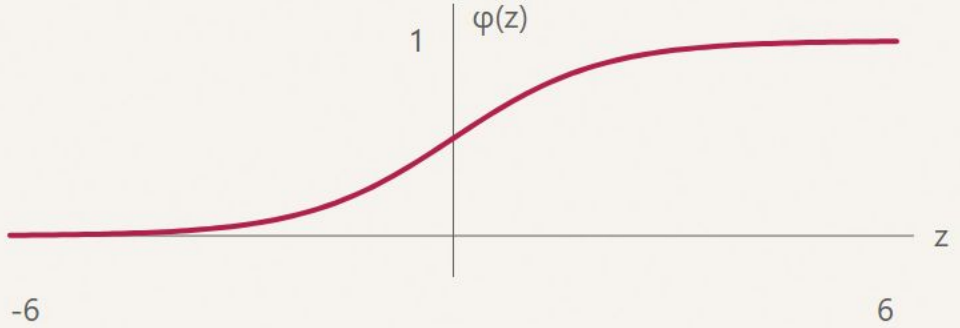


Activations shape both the features going forward and the derivatives coming back.

Sigmoid and tanh can saturate.

Sigmoid

$$\sigma(z) = 1 / (1 + e^{-z})$$

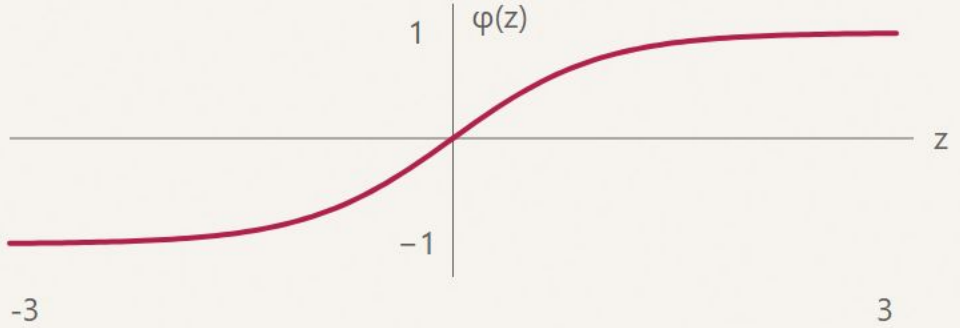


$$\sigma'(z) = \sigma(z)(1-\sigma(z)) \leq \tfrac{1}{4}$$

Range (0, 1) · binary probability / gates.

Tanh

$$\varphi(z) = \tanh(z)$$



$$\varphi'(z) = 1-\tanh^2(z)$$

Range (−1, 1) · centered, but still saturates.

Large |z| flattens both curves: local derivatives approach zero.

ReLU keeps the positive side open.

Avoid positive-side saturation—but watch for units that never open.

$$\mathrm{ReLU}(z) = \max(0, z)$$

$$\varphi'(z) = 0 \text{ for } z < 0;\ 1 \text{ for } z > 0$$

$$\text{Leaky ReLU}(z) = \max(\alpha z, z)$$

$\alpha = 0.1$ in the plotted comparison.



A persistently negative ReLU gets zero local gradient; leaky ReLU retains slope α.

Depth multiplies local derivatives.

A CHAIN OF SENSITIVITIES

$$\frac{\partial L}{\partial h_0} = \frac{\partial L}{\partial h_K} \prod_{\ell=1}^{K} \frac{\partial h_\ell}{\partial h_{\ell-1}}$$

Scalar chain; vector networks use ordered Jacobians.

VANISHING

$$0.5^{20} \approx 9.5 \times 10^{-7}$$

Almost no gradient reaches early layers.

EXPLODING

$$1.5^{20} \approx 3.3 \times 10^{3}$$

Small changes can create huge updates.

Set a reference. Then learn the mix.

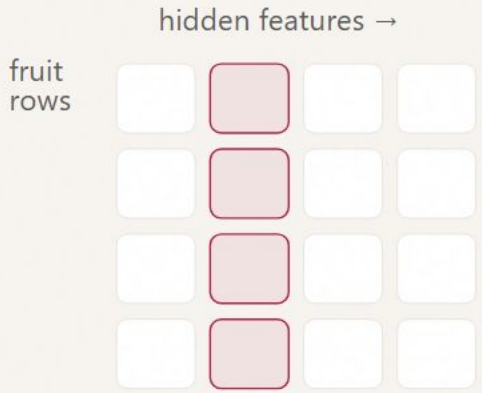
$$\hat{a} = \frac{a - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$u = \gamma \hat{a} + \beta$$

Hidden activations—not raw grams or centimeters. γ and β are learned.

Choose the normalization axes.

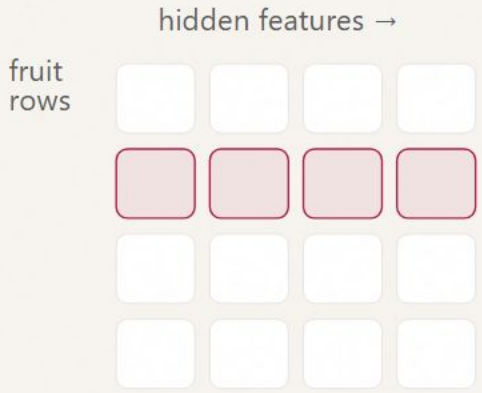
BATCHNORM



$\mu = 5 \cdot \sigma^2 = 5$

Same feature. Different examples.

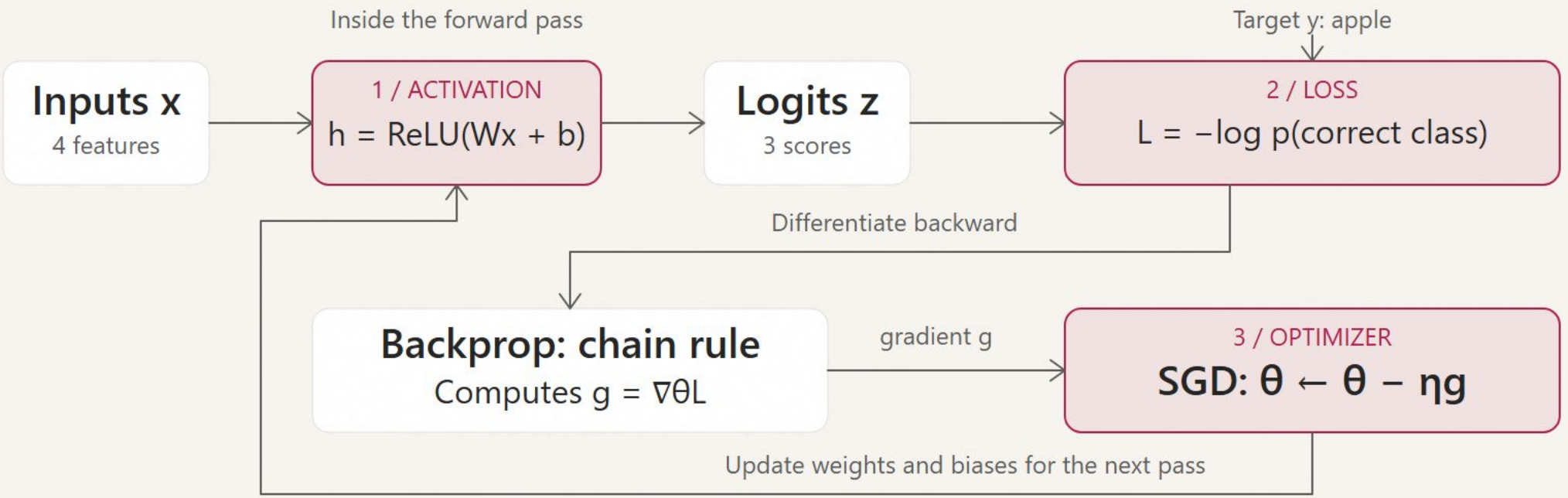
LAYERNORM



$\mu = 4.5 \cdot \sigma^2 = 1.25$

Same example. Different features.

Three functions. Three different jobs.



Now choose how training starts.

Optimizer, activation and loss are chosen. Make their signals usable.

UPDATE

SGD → memory → adaptive scale

SIGNAL

Activation + normalization

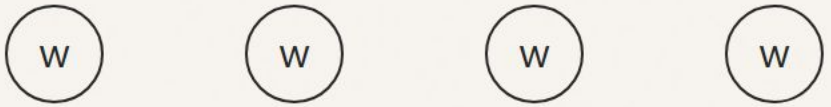
STARTING POINT

Break symmetry. Match the scale.

Final toolkit: initialization, preprocessing, diagnostics and controlled experiments.

Identical starts keep neurons redundant.

SAME WEIGHTS



Same outputs. Same gradient signals.

INDEPENDENT RANDOM WEIGHTS



Room to specialize.

Xavier balances the signal scale.

$$\mathrm{Var}(z) \approx n_{\mathrm{in}} \, \mathrm{Var}(W) \, \mathrm{Var}(x)$$

$$\mathrm{Var}(W_{ij}) = \frac{2}{n_{\mathrm{in}} + n_{\mathrm{out}}}$$

Linear / tanh-like layers · approximate assumptions · activation-specific gain

ReLU needs the He scale.



$$\mathrm{Var}(W_{ij}) = \frac{2}{n_{\mathrm{in}}}$$

He / Kaiming for ReLU. First layer: standard deviation √(2/4) ≈ 0.707.

Variance is not a sampling range.

Xavier or He chooses the scale σ. The sampler uses it differently.

Normal

$W \sim N(0, \sigma^2)$

Pass standard deviation σ, not variance σ^2.

Uniform

$W \sim U(-a, a)$

$a = \sqrt{3}\,\sigma$

Same variance σ^2.

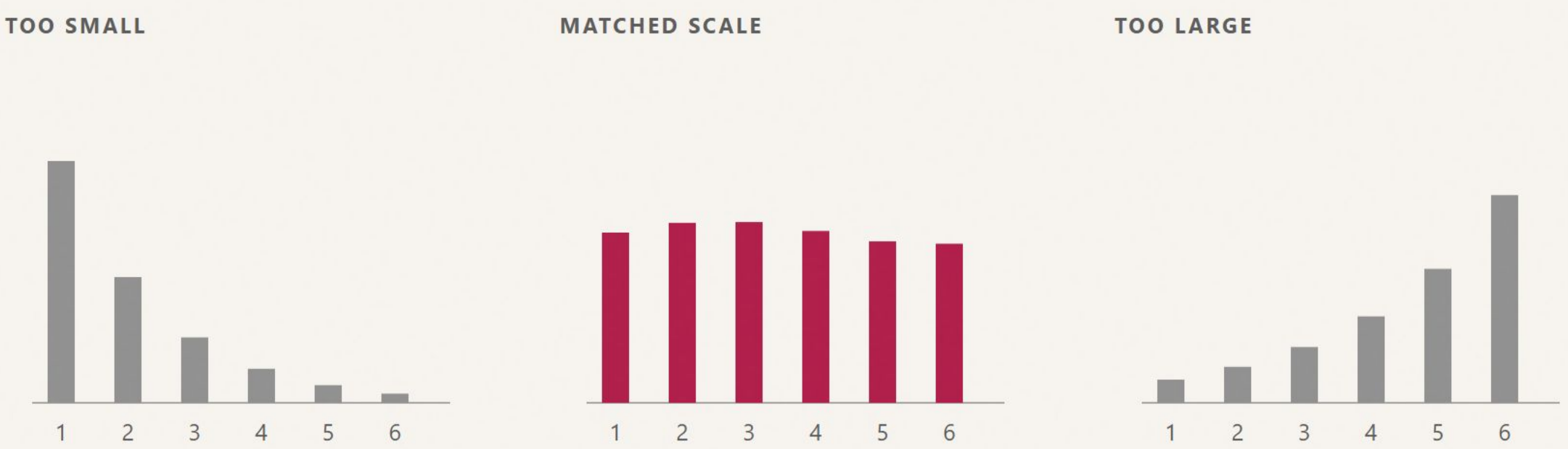
Orthogonal

$W = GQ$

Orthogonal directions; activation-specific gain G.

Example: He, fan-in 4 → σ ≈ 0.707; uniform limits ≈ ±1.225.

Inspect the signal—not just the seed.



Schematic magnitudes. Inspect activations and gradients.

Do not study from the final exam.

TRAIN	VALIDATION	TEST
700	150	150
Fit weights	Choose the recipe	Report after choices freeze

Illustrative split: 1,000 fruits. Fit preprocessing on training only.

Make one tiny batch learn first.

01

Data

Units. Order. Labels.

02

Objective

Shapes. Logits. Dtype.

03

Gradients

Finite? Nonzero?

04

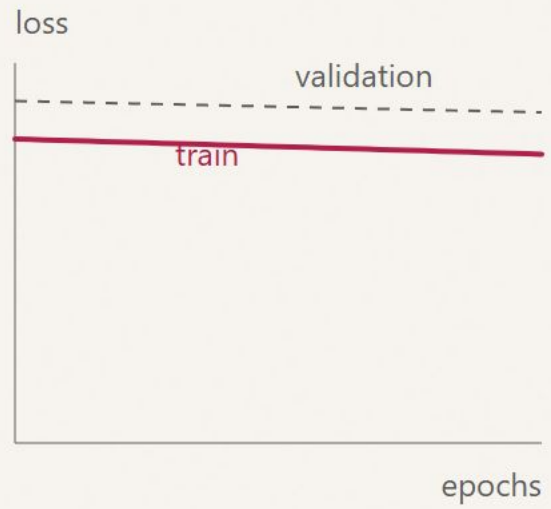
Update

Weights actually change?

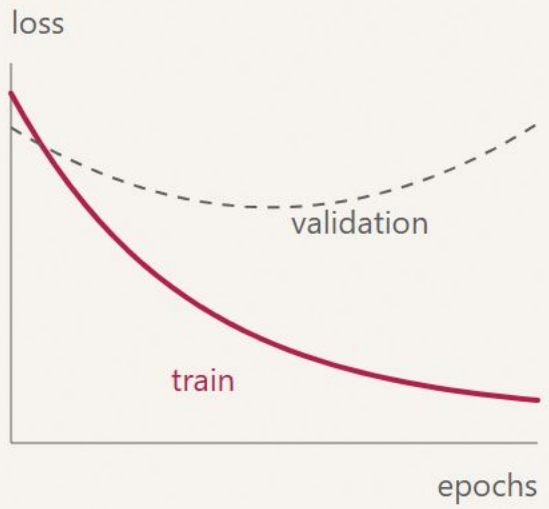
Flat loss? Trace these checkpoints on 32 fixed examples.

Read curves as hypotheses, not verdicts.

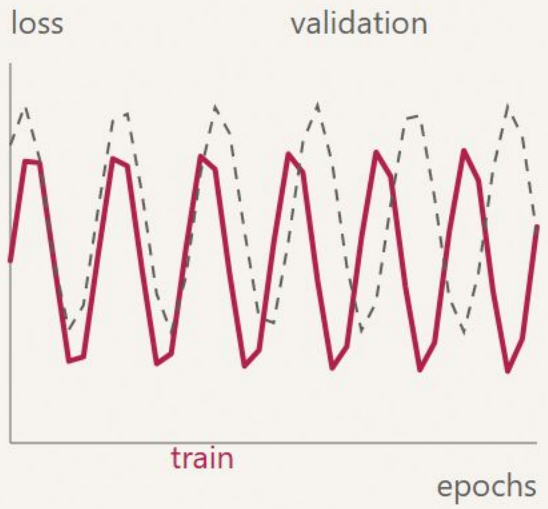
NOT LEARNING



OVERFITTING?



UNSTABLE



Illustrative patterns. Follow each hypothesis with a targeted experiment.

Match the safeguard to the symptom.

PLATEAU

Revisit η

Try learning-rate decay.

Warmup when early updates are unstable.

EXPLODING GRADIENTS

Clip the norm

Limit an oversized update.

Still investigate the cause.

OVERFITTING

Regularize

Weight decay / dropout.

Early-stop on validation.

Change one setting at a time. Compare validation at a fixed training budget.

Choose an intervention—not a buzzword.

A / COLLAPSING SIGNALS

Deep ReLU.

Inspect He scale.

B / ERRATIC INFERENCE

BatchNorm.

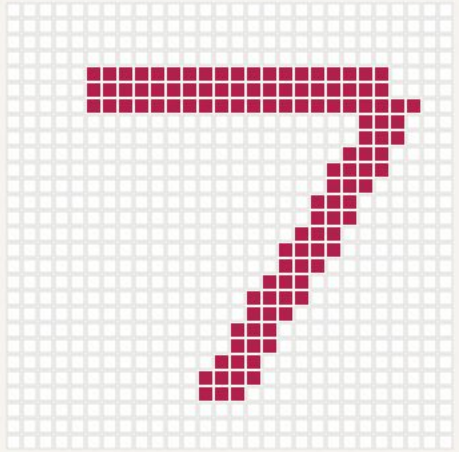
Check eval statistics.

C / IDENTICAL UNITS

Same start.

Break weight symmetry.

An image can become a vector, too.



Pixel = one brightness number.

$$28 \times 28 = 784$$

Read rows in a fixed order.

0–255 → divide by 255

10

Digit scores: 0–9

Build. Observe. Change one thing.

Start: a working vanilla SGD baseline.
Compare: tune η for each optimizer.
Save: the best validation checkpoint.

Next: RBFs → representation learning → autoencoders, CNNs, RNNs.